# Shadow Model Membership Inference Attack

Shokri et al., 2017 — Concept Explanation

## 1. The Core Question

The attack answers one specific question: **Was this data sample used to train the target model?** This is called a **Membership Inference Attack (MIA)**. If an attacker can answer this, they can leak private training data — such as medical records or financial data.

## 2. Why It Works — The Key Intuition

ML models behave differently on data they were trained on versus data they have never seen:

- **Training data (members):** higher confidence, sharper softmax output → e.g., [0.01, 0.95, 0.02, ...]
- **Unseen data (non-members):** lower, flatter confidence → e.g., [0.15, 0.42, 0.18, ...]

This behavioral difference — a symptom of **overfitting** — is precisely what the attack exploits.

## 3. The Problem & The Solution

The attacker has only **black-box access** to the target model: they can query it and receive softmax outputs, but they do not know what data it was trained on. So they cannot directly learn 'what does a member look like?'

**Solution:** Train *shadow models* — copies of the target model trained on known data — to simulate its behavior. Since ground truth (member vs. non-member) is known for the shadow models, labeled training data can be generated for an attack classifier, which then generalises to the real target.

## 4. Full Pipeline (5 Stages)

### Stage 1 — Build a Shadow Pool

```
CIFAR-10 has 50,000 training samples
Target model used 6,000 of them  →  attacker excludes these
~44,000 remaining samples  →  'shadow pool'  (attacker's independent data)
```

### Stage 2 — Train Shadow Models

The attacker trains **4 shadow models** using the same SmallCNN architecture as the target. For each shadow model:

- Randomly pick 3,000 samples from the pool → **'in' (members)**
- Next 3,000 samples → **'out' (non-members)**

- Train the shadow model on the 'in' samples for 80 epochs

### Stage 3 — Collect Labeled Attack Training Data

After training each shadow model, run both 'in' and 'out' samples through it to extract softmax confidence vectors and assign membership labels:

```
softmax('in'  sample) → [0.01, 0.95, 0.01, ...]  → label = 1  (member)
softmax('out' sample) → [0.15, 0.42, 0.10, ...]  → label = 0  (non-member)

4 shadows × 6,000 samples = 24,000 labeled (softmax_vector, is_member) pairs
```

### Stage 4 — Train Per-Class Attack Classifiers

10 binary **AttackMLP** classifiers are trained — one per CIFAR-10 class. Each MLP maps a 10-dim softmax vector to a membership logit.

**Why per-class?** Each class has a unique confidence signature. Training class-specific classifiers exploits this structure for higher precision.

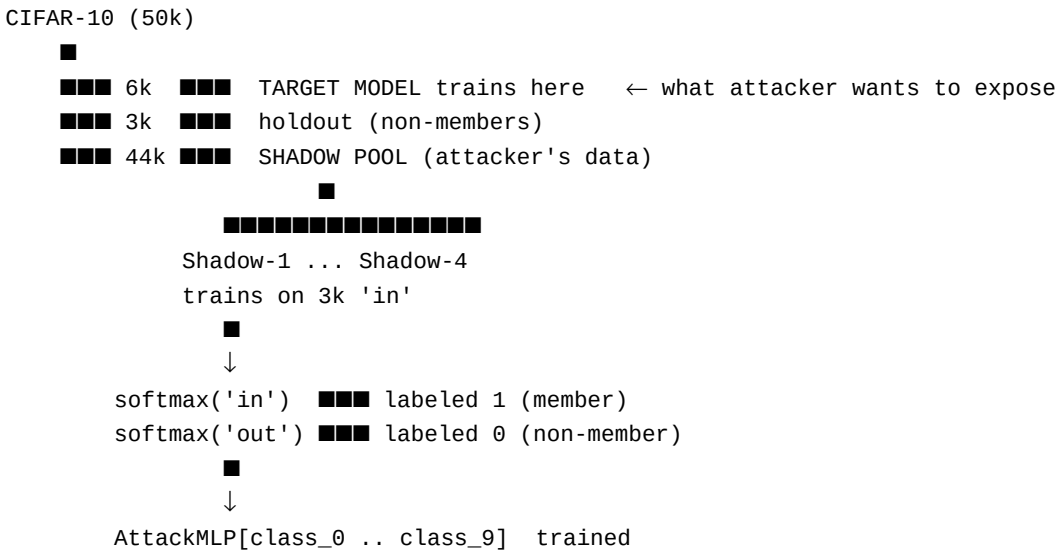
```
Input:  10-dim softmax vector
Output: scalar logit  →  P(is_member)
Architecture: Linear(10→64) → ReLU → Linear(64→32) → ReLU → Linear(32→1)
```

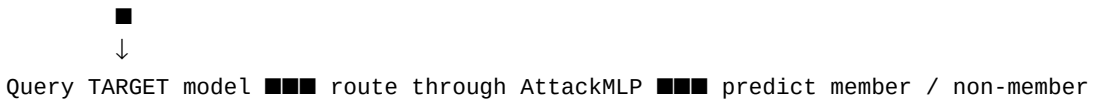
### Stage 5 — Attack the Real Target Model

The attacker queries the **actual target model** with members and non-members, then routes each softmax output through the matching class-specific AttackMLP:

- 6,000 **members** (target's training set) → ground truth = 1
- 3,000 **non-members** (holdout) → ground truth = 0
- If logit > 0 → predict 'member'

## 5. Data Flow Diagram






## 6. Evaluation Metrics

| Metric | What It Means |
|---|---|
| Accuracy | % of samples correctly classified as member / non-member |
| AUC-ROC | 0.5 = random guess,  1.0 = perfect attack |
| Attacker Advantage | 2 × (TPR – FPR);  0 = no better than random |
| Recall (TPR) | % of actual members correctly identified |
| FPR | % of non-members wrongly flagged as members |

## 7. Summary

The shadow model attack **transfers knowledge** from shadow models (where ground truth is known) to infer membership in the real target model. The shadow models act as a proxy to generate labeled training data for the attack classifiers, which then generalise to the real model because:

- Both target and shadow models share the same **architecture**
- Both are trained on data drawn from the **same distribution** (CIFAR-10)
- Overfitting creates a **consistent, exploitable signal** in the softmax outputs

*Source file: attacks/shadow_model_attack.py — adversarial-privacy-attacks-defense project*